



Analysis & Development of Algorithms

II

Anton Kovantsev

https://t.me/pop_antonij

Algorithms for unconstrained nonlinear optimization.

Direct methods

- Bazara and Shetty. Nonlinear programming: Theory and algorithms
- Bazara, Jarvis and Sherali. Linear Programming and Network Flows
- Bonnans, Gilbert, Lemarechal, and Sagastizabal. Numerical Optimization.
- Theoretical and Practical Aspects
- Ben-Tal and Nemirovski. Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications
- Nocedal J., Wright Stephen J. Numerical Optimization
- and many others.

Optimization problem

Optimization methods are the methods for finding optimal (in some sense) solutions for mathematical models

Various applications, including in Machine Learning and Data Mining

A mathematical model can be usually expressed via an objective function $f = f(x)$, where x is generally a multidimensional vector, or via an optimality criterion (with constraints or without).

Optimization problem

To solve the optimization problem $f(x) \rightarrow \max_{x \in Q} (\min_{x \in Q})$ means to find $x^* \in Q$, where Q is the region of acceptability, such that $f(x)$ reaches the maximal (minimal) value at x^* .

Notation: $x^* = \arg \max_{x \in Q} (\min_{x \in Q}) f(x)$. If x^* is known, one can clearly find $f(x^*)$

- **Local** (simpler) and **global** (more complex) optimization (coincide for some classes of $f(x)$)
- The **max**-problem becomes the **min**-problem if one considers $-f(x)$

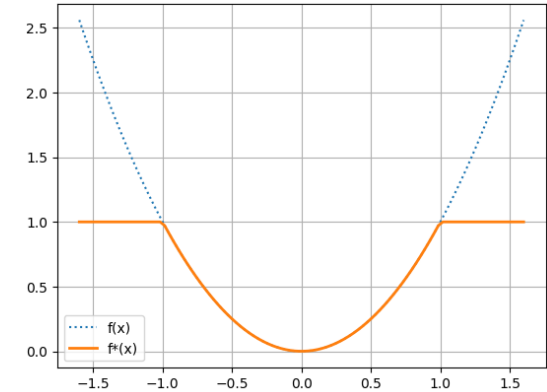
Problem types

Q, the region of acceptability, may be

- not defined (or reduced to be not defined) (unconstrained optimization);

for example, minimize $f(x) = x^2$ for $x \in [-1, 1]$ (i.e. Q is defined) = minimize (without any constraints) the function

$$f^*(x) = \begin{cases} x^2 & x \in [-1, 1] \\ 1 & x \notin [-1, 1] \end{cases}$$



- defined via a system S of linear or non-linear equations or inequalities (constrained optimization)

f or S (non-)linear	$\rightarrow$	(non-)linear programming
		(non-)linear optimization

For purposes of Machine Learning and Data Mining, we are interested now in
unconstrained non-linear optimization.

Methods for minimization

Information about $f \rightarrow$ direct, first-order and second-order methods

Direct, or zero-order, methods only values of f (but not those of its derivatives) are used.

+ available for a wide class of functions; - slow convergence

First-order methods values of f and f' are used (gradient methods).

+ relatively fast convergence; - necessity to know the analytic expression of f and f'

Second-order methods values of f , f' and f'' are used (Newtonian methods).

+ fast convergence; - necessity to know the analytic expression of f , f' and f''

Exhaustive search

Let $f(x):[a,b]\rightarrow\mathbb{R}$.

Approximately solve the optimization problem
 $f(x)\rightarrow\min x\in[a,b]$ by finding x^* with error $\epsilon>0$.

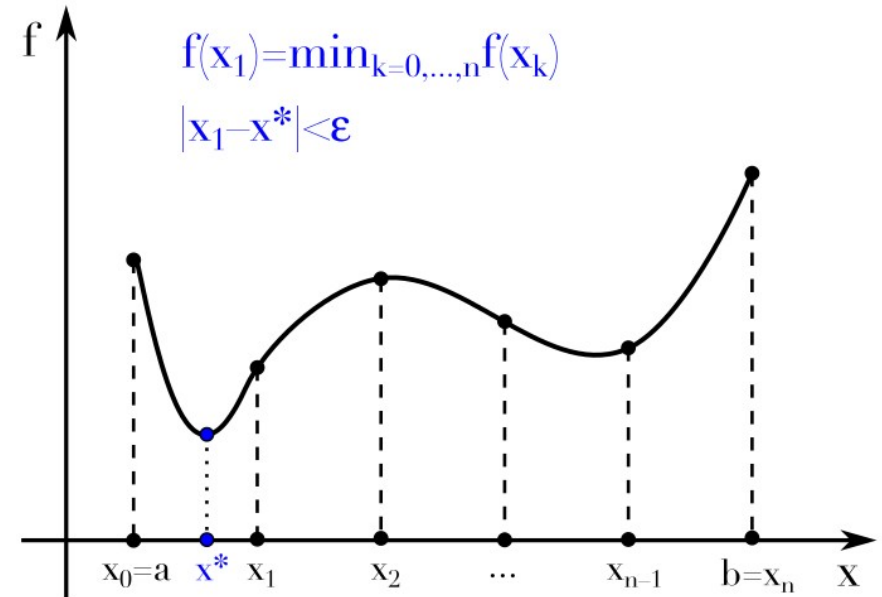
Algorithm

1. Consider the following points $\in[a,b]: x_k = a + k \frac{b-a}{n}$,

$k=0,\dots,n$, n is chosen so that $\frac{b-a}{n}\leq\epsilon$.

2. Calculate $f(x_k), k\in\{0,\dots,n\}$, find $\hat{x}: f(\hat{x}) = \min f(x_k)$.

3. Then $|\hat{x} - x^*| \leq \epsilon$, i.e. $\hat{x}$ is an approximant to x^* .



This method is very slow but may be used for finding initial approximations.

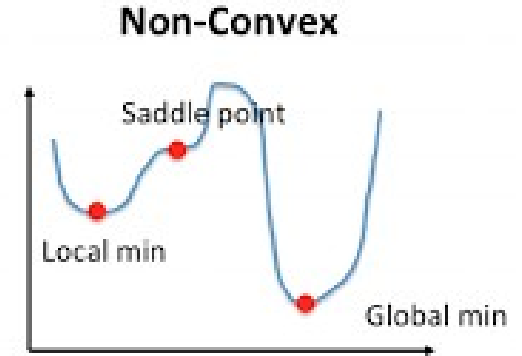
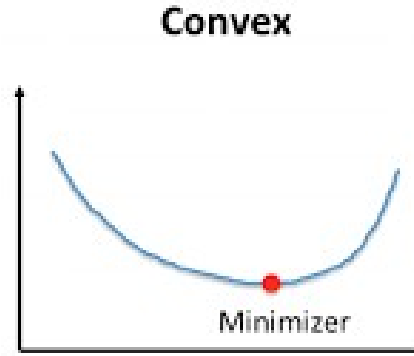
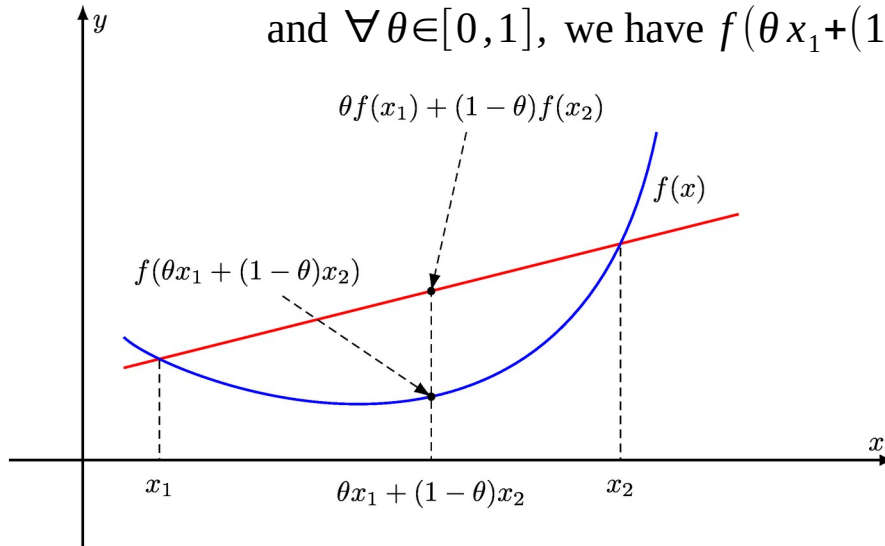
Dichotomy method

Let $f(x):[a_0, b_0] \rightarrow \mathbb{R}$ and be convex.

Approximately solve the optimization problem $f(x) \rightarrow \min_{x \in [a_0, b_0]}$ by finding x^* with error $\epsilon > 0$.

What is „convex“

A function $f:\mathbb{R}^n \rightarrow \mathbb{R}$ is convex if its domain is a convex set and for all x_1, x_2 in its domain, and $\forall \theta \in [0, 1]$, we have $f(\theta x_1 + (1 - \theta)x_2) \leq \theta f(x_1) + (1 - \theta)f(x_2)$



Dichotomy method

Let $f(x):[a_0, b_0] \rightarrow \mathbb{R}$ and be convex.

Approximately solve the optimization problem $f(x) \rightarrow \min_{x \in [a_0, b_0]}$
by finding x^* with error $\epsilon > 0$.

Algorithm

1. Calculate

$$x_1 = \frac{a_0 + b_0 - \delta}{2}, x_2 = \frac{a_0 + b_0 + \delta}{2}, 0 < \delta < \epsilon,$$

then $f(x_1)$ and $f(x_2)$.

2. Reduce the indeterminacy segment down to the segment $[a_1, b_1]$:

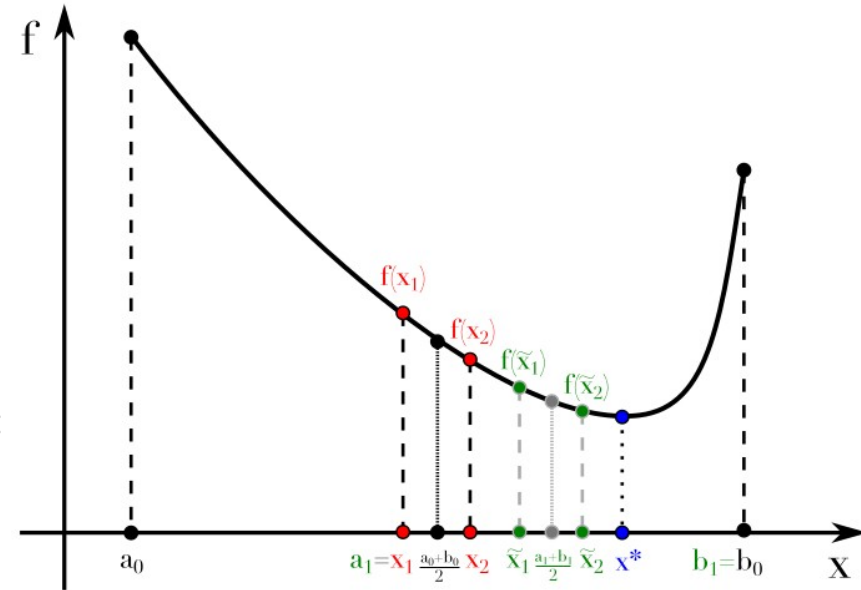
if $f(x_1) \leq f(x_2)$, then $a_1 = a_0$ and $b_1 = x_2$;

$a_1 = x_1$ and $b_1 = b_0$, otherwise.

3. Furthermore, by analogous formulas find x_1 and $x_2 \in [a_1, b_1]$.

repeat the reducing procedure down to a segment $[a_2, b_2]$, etc.

4. The search is stopped if at the current k-th iteration it holds that $|a_k - b_k| < \epsilon$; $x^* \in [a_k, b_k]$.



Golden section method

It allows to find the minimum in $[a_0, b_0]$ with less computations.
Essentially, it is the dichotomy method with a special choice of δ .

Let $f(x):[a_0, b_0] \rightarrow \mathbb{R}$ and be convex.

Approximately solve the optimization problem $f(x) \rightarrow \min x \in [a_0, b_0]$
by finding x^* with error $\epsilon > 0$.

1. Calculate

$$x_1 = a_0 + \frac{3 - \sqrt{5}}{2}(a_0 + b_0), x_2 = b_0 + \frac{\sqrt{5} - 3}{2}(a_0 + b_0), \left(\frac{x_1 + x_2}{2} = \frac{a_0 + b_0}{2} \right)$$

then $f(x_1)$ and $f(x_2)$. (the first iteration requires to calculate two points and two values of f).

2. Reduce the indeterminacy segment down to the segment $[a_1, b_1]$:

if $f(x_1) \leq f(x_2)$, then $a_1 = a_0, b_1 = x_2, x_2 = x_1$;

$a_1 = x_1$ and $b_1 = b_0, x_1 = x_2$ otherwise.

3. On the forthcoming iterations, calculate one point and one corresponding value of f :

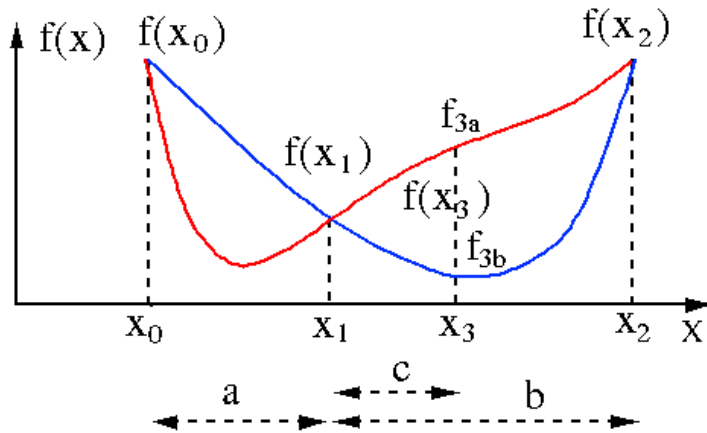
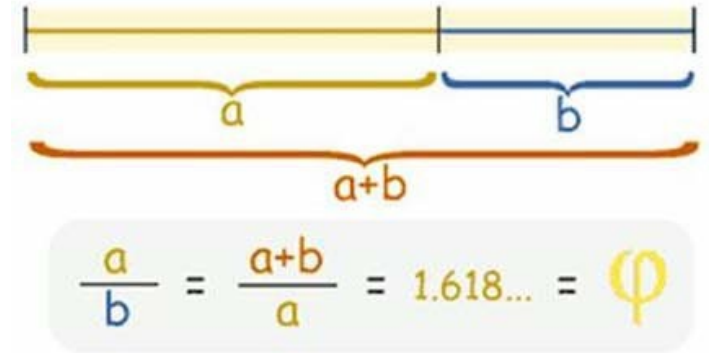
in the former case x_1 and $f(x_1)$, and in the latter one x_2 and $f(x_2)$.

4. The search is stopped if at the current k -th iteration it holds that $|a_k - b_k| < \epsilon; x^* \in [a_k, b_k]$.

Golden section method

The golden-section search chooses the spacing between these points in such a way that these points have the same proportion of spacing as the subsequent triple

This choice of interval ratios is the only one that allows the ratios to be maintained during an iteration.



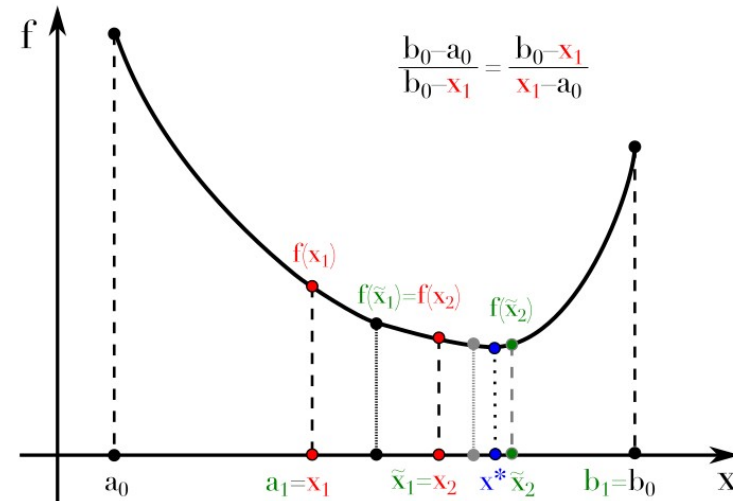
$$\frac{c}{a} = \frac{a}{b}$$

We want $\frac{c}{b-c} = \frac{a}{b}$

$$\left(\frac{b}{a}\right)^2 - \frac{b}{a} = 1 \Rightarrow \frac{b}{a} = \phi = \frac{1+\sqrt{5}}{2}$$

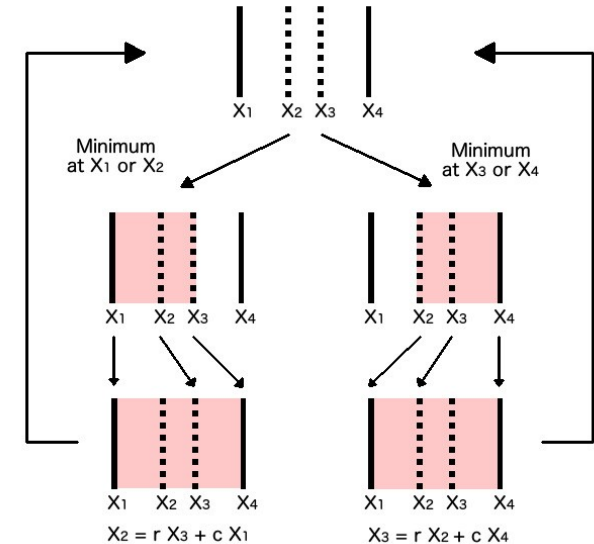
Some magic:

$$\left(\frac{1}{\phi}\right)^2 = 1 - \frac{1}{\phi} = \frac{3-\sqrt{5}}{2}$$



Golden section method

- 1) Specify the function to be minimized, $f(x)$, the interval to be searched as $\{X_1, X_4\}$, and their functional values F_1 and F_4 .
- 2) Calculate an interior point and its functional value F_2 . The two interval lengths are in the ratio $c : r$ or $r : c$ where $r = \phi - 1$; and $c = 1 - r$, with ϕ being the golden ratio.
- 3) Using the triplet, determine if convergence criteria are fulfilled. If they are, estimate the X at the minimum from that triplet and return.
- 4) From the triplet, calculate the other interior point and its functional value. The three intervals will be in the ratio $c : cr : c$.
- 5) The three points for the next iteration will be the one where F is a minimum, and the two points closest to it in X .
- 6) Go to step 3.



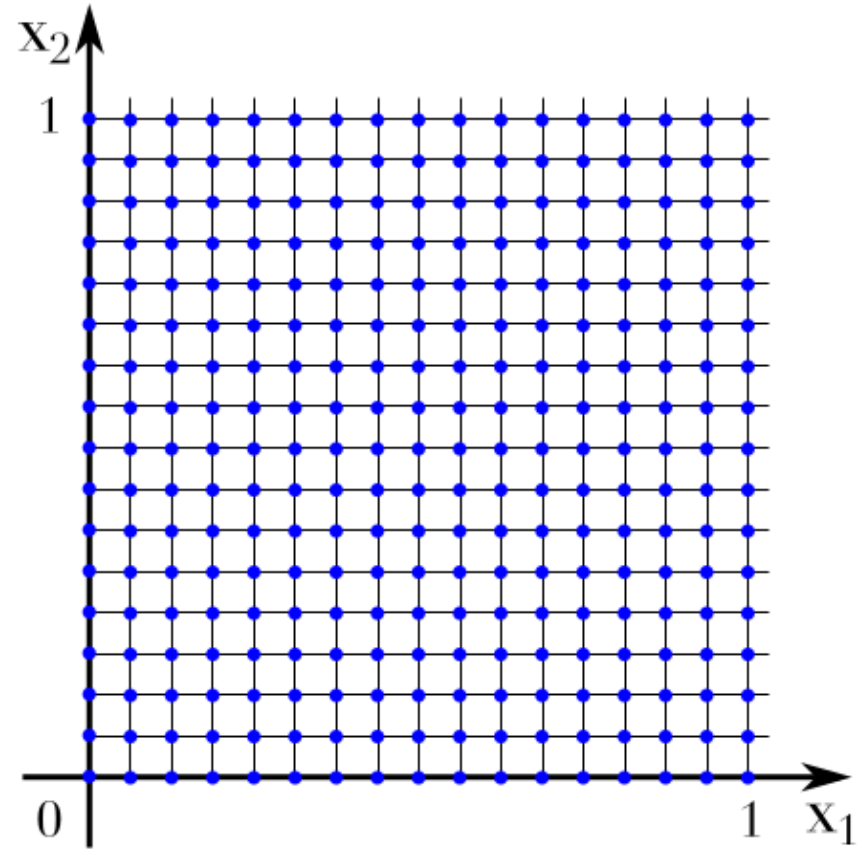
Multidimensional exhaustive

Let $f(x): D \rightarrow \mathbb{R}$, where $x = (x_1, x_2)$, $D = \{[0, 1] \times [0, 1]\}$.

Approximately solve the optimization problem

$f(x) \rightarrow \min_{(x, y) \in D}$ with error $\epsilon > 0$.

If $\hat{x}$ is an approximant for x^* one chooses the sufficient grid, e.g., the condition $|\hat{x} - x^*| \leq \epsilon$.



Gauss method

Coordinate descent

In each iteration, the minimization is carried out only with respect to one vector component of the variable x . We consider $x = (x_1, x_2)$.

Algorithm

Let $x^0 = (x_1^0, x_2^0)$ be the initial approximation.

1. In the first iteration, find the minimum point of f as a function of the first variable while others are fixed,

i.e. $x_1^1 = \operatorname{argmin}_{x_1} f(x_1, x_2^0)$, to get a new point $x_1 = (x_1^1, x_2^0)$.

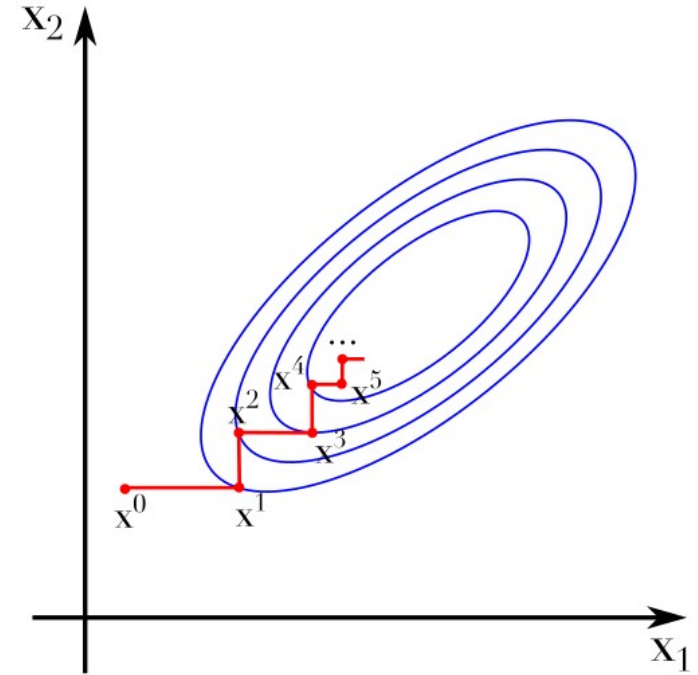
2. Furthermore, using x_1 , find the minimum point

by varying only the second variable: i.e. find $x_2^1 = \operatorname{argmin}_{x_2} f(x_1^1, x_2)$ and a new point $x_2 = (x_1^1, x_2^1)$.

3. Start searching again by the first variable, etc.

4. The search is stopped under one of the following criteria:

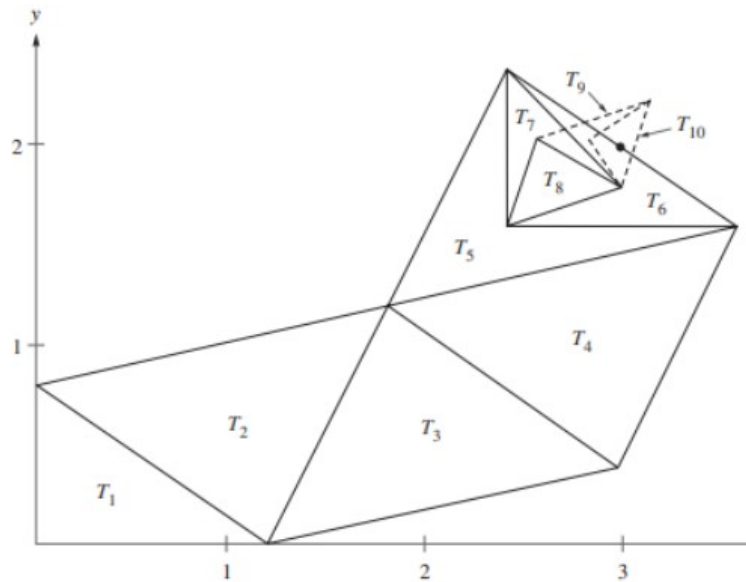
$$|x_i^{k+1} - x_i^k| \leq \epsilon, \quad i=1, 2, \quad \text{or} \quad |f(x_{k+1}) - f(x_k)| \leq \epsilon.$$



The method is simple but problems may appear when the level lines of f are close to the line $x_1 = x_2$. If the initial approximation is on $x_1 = x_2$, then the process gets stuck.

Nelder-Mead method

It uses simplexes in $\mathbb{R}^n$. In our case ($n = 2$), they are triangles.



This is an heuristic approach which can stuck in a local minima or converge to a non-stationary point.

Parameters: reflection coefficient $\alpha > 0$ (usually $\alpha = 1$),
shrinking coefficient $0 < \beta < 1$ (usually $\beta = 0.5$),
expansion coefficient $\gamma > 1$ (usually $\gamma = 2$).

Step 1 (preparation). Choose three points $x_i = (x_i^{(1)}, x_i^{(2)})$, $i = 1, 2, 3$ for the initial simplex (triangle in our case).

Calculate $f_1 = f(x_1)$, $f_2 = f(x_2)$, $f_3 = f(x_3)$.

Step 2 (sorting). Choose three points from the simplex vertices:
 x_h with the largest value of f_h ,
 x_g with the second-large value of f_g and x_l with the smallest value of f_l .

→ The goal of the forthcoming manipulations is to decrease f_h at least.

Step 3 (gravity centre). Find the gravity centre for all point except

$$x_h: x + c = \frac{1}{2} \sum_{i \neq h} x_i.$$

Nelder-Mead method

Step 4 (reflection). Reflect the point x_h with respect to x_c with the coefficient α (for $\alpha=1$ is the central symmetry): $x_r = (1 + \alpha)x_c - \alpha x_h$. Calculate $f_r = f(x_r)$.

Step 5 (expansion). Furthermore, check how the function values are decreased:

If $f_r < f_l$, then the direction is right, we can expand: calculate $x_e = (1 - \gamma)x_c + \gamma x_r \wedge f_e = f(x_e)$.

If $f_e < f_r$, the the simplex can be expanded: set $x_h := x_e$ and go to Step 7.

If $f_r < f_e$, then we moved too far: set $x_h := x_r$ and go to Step 7.

If $f_l < f_r < f_g$, then the choice of the new point is good (the new one is better than previous two): set $x_h := x_r$ and go to Step 7.

If $f_g < f_r < f_h$, then exchange $x_r \leftrightarrow x_h$ and $f_r \leftrightarrow f_h$. go to Step 6.

If $f_h < f_r$, then go to Step 6.

As a result, $f_l < f_g < f_h < f_r$.

Step 6 (shrinking). Calculate $x_s = \beta x_h + (1 - \beta)x_c$ and $f_s = f(x_s)$.

If $f_s < f_h$, then set $x_h := x_s$ and go to Step 7.

If $f_s > f_h$, then the initial points are the best. Shrink the simplex globally: $x_i := x_l + (x_i - x_l)/2, i \neq l$.

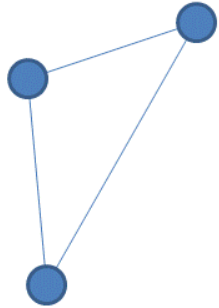
Step 7 (convergence check). Check the mutual closeness of the simplex vertices

(for example, by estimating the deviation of the vertices), i.e. the closeness to the minimum point.

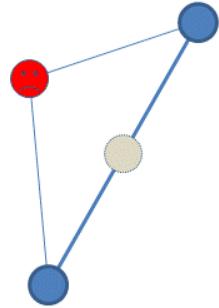
If the required precision is not achieved, go to Step 2.

Nelder-Mead method

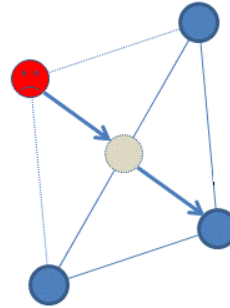
Nelder-Mead algorithm



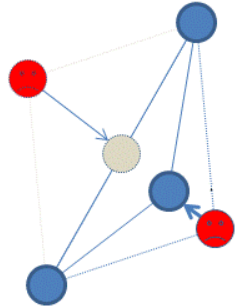
1: Initial Simplex



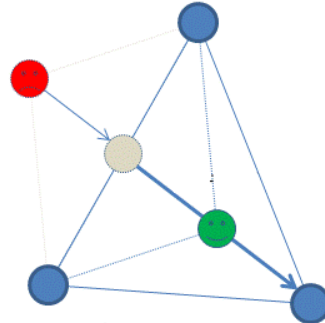
2: Center of gravity
(without the worst point)



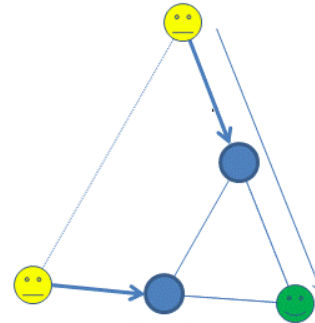
3: Reflection



4a: Contraction



4b: Expansion



5: Shrinkage

Coefficients:

α — reflection,

γ — expansion,

ρ — contraction

σ — shrink

Standard values are:

$\alpha = 1$,

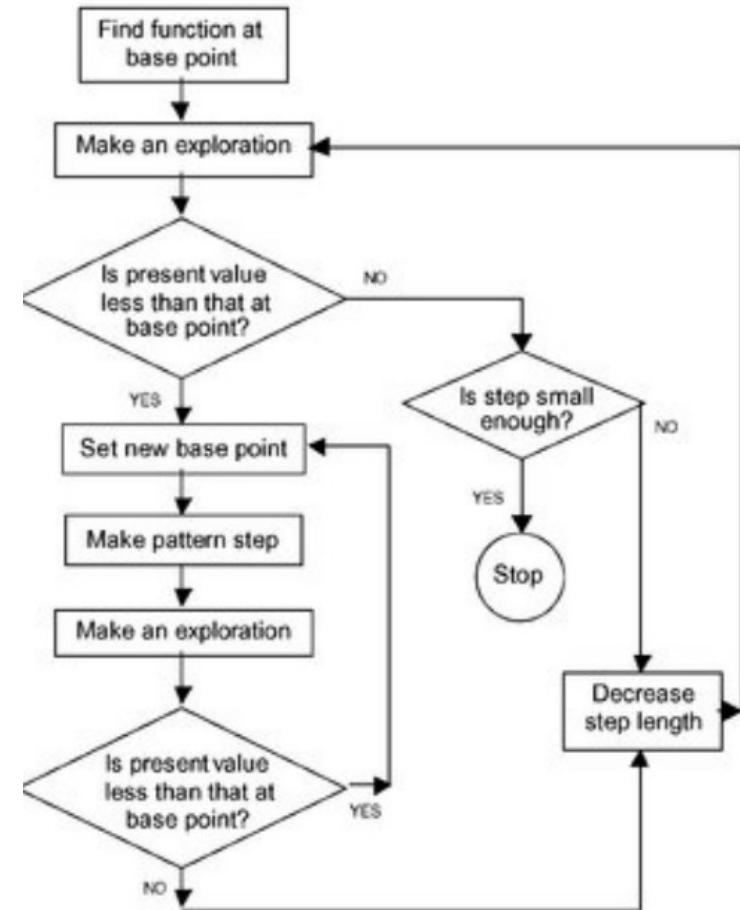
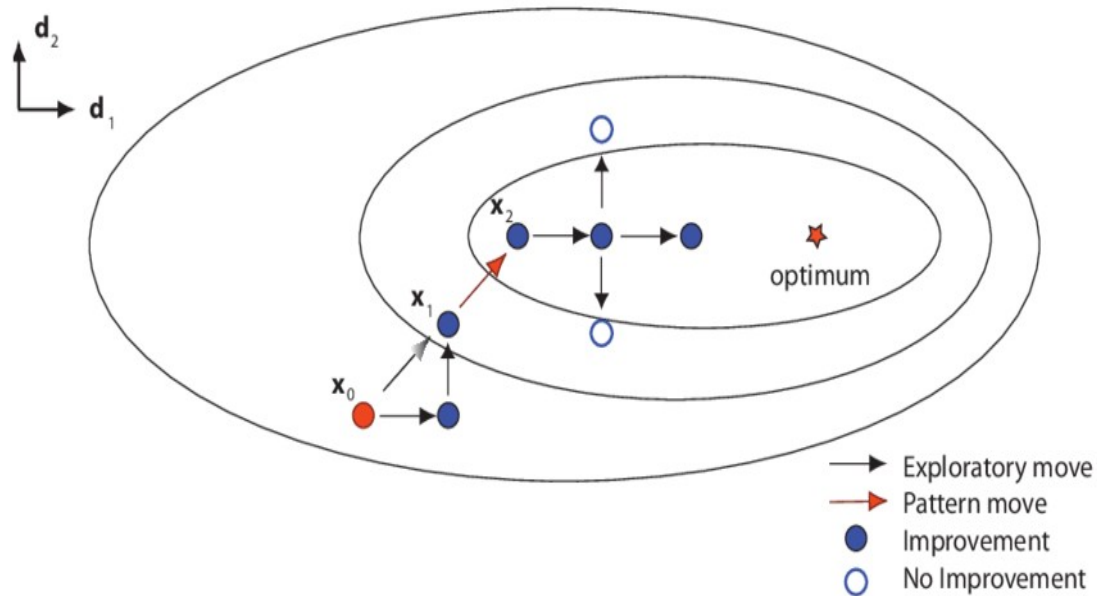
$\gamma = 2$,

$\rho = 1/2$

$\sigma = 1/2$.

Hooke-Jeeves Algorithm

Pattern search



Hooke-Jeeves Algorithm

The Steps of Hooke-Jeeves

0. Assign x_1 , x_2 , h_i
1. Cast about your original point to find the best point in its neighborhood. Test your initial point in one dimension only, for example by testing $(x_1 + h_i, x_2)$ and $(x_1 - h_i, x_2)$. Save the best value of x_1 .
2. Travel along that vector until you stop seeing improvement.
3. Repeat steps 1 and 2 until step 1 returns a vector of $(0, 0)$.
4. Reduce the interval $h_i := h_i * n$ ($0 < n < 1$)
5. Repeat steps 1-4 until the interval h_i is small enough to achieve the required tolerance.

Random search

Albert Einstein: *"God does not play dice with the universe."*

Niels Bohr: *"Einstein, stop telling God what to do."*

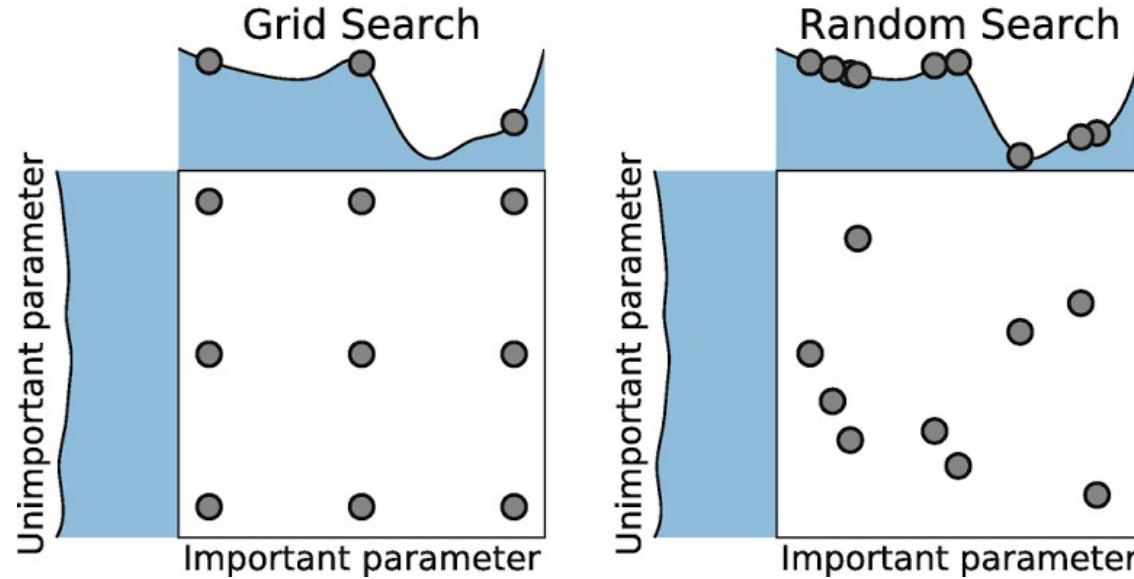
This search goes on sequentially on each parameter and refines iteratively on the best guesses from the last sequence. The pattern can be a grid (factorial) search of all parameters, a sequential search on each parameter, or a combination of both. The method was developed to screen the experimental conditions in chemical reactions.

Algorithm:

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be the fitness or cost function which must be minimized. Let $x \in \mathbb{R}^n$ designate a position or candidate solution in the search-space. The basic RS algorithm can then be described as:

- 1) Initialize x with a random position in the search-space.
- 2) Until a termination criterion is met (e.g. number of iterations performed, or adequate fitness reached), repeat the following:
 - 3) Sample a new position y from the hypersphere of a given radius surrounding the current position x (see e.g. Marsaglia's technique for sampling a hypersphere.)
 - 4) If $f(y) < f(x)$ then move to the new position by setting $x = y$

Random search



Random search has been used in artificial neural network for hyper-parameter optimization.

If good parts of the search space occupy 5% of the volume the chances of hitting a good configuration in search space is 5%. The probability of finding at least one good configuration is above 95% after trying out 60 configurations:

$$1 - 0.95^{60} \approx 0.953 > 0.95 \text{ (making use of the counterprobability).}$$



АНТОН КОВАНЦЕВ
pop.antonij@gmail.com
https://t.me/pop_antonij